Phase 6 | System Design, Performance & Scalability

Separate what exists now from sensible future changes.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Current design

A single web process serves Jinja pages and JSON, loads model/artifacts/config at import time, holds station and graph structures in memory, and computes predictions on demand. The current dataset is a static synthetic file used by offline training; it is not scanned per request. There is no database or background worker.

2. Likely bottlenecks (measure first)

- Cold start: reading station/operations data, deserializing model, and building graph during imports.
- Memory: each Uvicorn worker has its own model, station tables, and graph.
- Full profile calculation: 19 predictions per request plus several pandas DataFrame constructions and graph search.
- JSON payload: travel times to many destinations for each UI refresh.
- Threadpool capacity: requests can wait if synchronous calculations saturate the shared pool.

Measure startup wall time, memory per worker, p50/p95 endpoint latency, CPU, request concurrency, feature-generation time, model prediction time, and route-search time. Use representative station selections and cold/warm processes. Avoid optimizing based only on guesswork.

3. Low-risk optimizations

- Cache immutable station metadata, precomputed feature skeletons, and route result maps by source station if graph and weights are static.
- Vectorize all 19 hourly feature rows into one DataFrame and one batch `predict` call rather than 19 one-row calls.
- Avoid rebuilding `stations_meta` from DataFrame on every dashboard request; precompute once.
- Return travel times only when requested or cache all-source routes; evaluate payload size and client needs.
- Keep training out of web startup; use committed model artifacts or a separate controlled build pipeline.
- Pin model/library versions and record artifact metadata.

4. Scaling paths

Small public demo: one Uvicorn process/worker, immutable local artifacts, autosleep acceptable; add startup health diagnostics and smoke test.

Moderate read traffic: multiple worker processes behind platform routing; confirm memory per worker and cold-start behavior; cache static graph outputs. Add request/latency/error monitoring.

Observed data ingestion: move observations and user-managed data to PostgreSQL; validate schema and timezones; use ingestion jobs and audit metadata.

Expensive retraining: separate batch pipeline/worker from API process; publish versioned model artifacts atomically; API loads known-good version and health reports active artifact ID.

Heavy forecasting or scenario simulation: background queue only if measured requests exceed latency budget; use job status IDs and persistence for results. Avoid adding distributed components to solve an unmeasured problem.

5. Model-serving boundaries

Current inference is in-process. Benefits: simple, low network overhead. Risks: model memory multiplies across web workers; deploy of API and model is coupled; one slow workload uses web concurrency. A dedicated model service is justified only when independent scaling/versioning or resource isolation is needed. A batch forecast cache may be simpler because dashboard slots are repeated station/date/time combinations.

6. Data storage evolution

CSV and JSON are reasonable read-only static inputs. To accept observed counts, define a provenance-rich schema: station ID, timestamp/time zone, count definition (entry/exit/transfer), source, quality flag, revision, and aggregation window. Use relational constraints and migrations; never silently mix synthetic and observed targets. Keep raw data immutable and derive features reproducibly.

7. Reliability

Add `/health/live` for process liveness and `/health/ready` to confirm model/metadata/data are present and compatible. Validate key artifacts at startup and log actionable errors. Ensure API can return explicit model version and `data_basis`. Graceful fallback should not produce plausible-looking results when critical model files are invalid. Render logs should exclude sensitive request payloads if any are later introduced.

8. Routing evolution

The current static graph is appropriate for a small network. GTFS or official route data could create edge tables with direction, travel time, service calendars, and transfer pathways. Time-dependent routing would require edge costs depending on departure time, service headways, and disruptions; ordinary static Dijkstra would no longer capture waiting time correctly. A graph library is optional, not automatically better.

9. Suggested benchmark plan

Create fixed test cases and measure: import/startup, one hourly prediction, 19-hour profile, one API response with route results, concurrency at 1/5/20/50 users, and memory at each worker count. Keep hardware/runtime/version constant. Set an explicit latency goal, compare before/after, and verify numeric output unchanged. Report limits honestly rather than claiming “scales” without measurements.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.